Home Care Optimised Resource Allocation Problem (HCORAP)

Problem Description, Mathematical Models, and Dataset

Contents

1 Problem Description **3**
1.1 Input Data . . . 3

2 Illustrative Example **3**

3 Main Constraints **4**

4 Integer Linear Programming (ILP) Model **5**
4.1 Decision Variables . . . 5
4.2 Objective Function . . . 5
4.3 Constraints . . . 5

5 SAT / MaxSAT Model **6**
5.1 Boolean Variables . . . 6
5.2 Hard Clauses . . . 7
5.3 Soft Clauses (Objective Function) . . . 8
5.4 Stratified Variant . . . 8

6 Optimisation Approaches **9**
6.1 Non-Incremental SAT . . . 9
6.1.1 Top-Down Linear Search . . . 9
6.1.2 Bottom-Up Linear Search . . . 9
6.1.3 Binary (Dichotomic) Search . . . 9
6.1.4 Cardinality Encoding . . . 10
6.1.5 Key Properties . . . 10
6.2 Incremental SAT . . . 10
6.2.1 Assumption-Based Bound Tightening . . . 10
6.2.2 Clause-Addition Linear Search . . . 11
6.2.3 Application to HCORAP . . . 11
6.2.4 Key Properties . . . 12
6.3 MaxSAT . . . 12
6.3.1 Formulation . . . 12
6.3.2 Internal Algorithms of MaxSAT Solvers . . . 12
6.3.3 Stratified Optimisation in HCORAP . . . 13
6.3.4 DIMACS WCNF Format . . . 13
6.3.5 Key Properties . . . 13
6.4 Comparison of Approaches . . . 13

7 Dataset **14**
7.1 Overview 14
7.2 Instance Generation 14
7.3 Instance File Format 15
7.4 Instance Statistics 15

1 Problem Description

The **Home Care Optimised Resource Allocation Problem (HCORAP)** arises in the context of home care service companies that assign care workers (agents) to perform services for users (patients/clients) over a weekly planning horizon.

A home care company manages a set of *agents* $\mathcal{A} = \{a_1, \ldots, a_A\}$ who visit a set of *users* $\mathcal{U} = \{u_1, \ldots, u_U\}$. Each user requires a set of *services* $\mathcal{S} = \{s_1, \ldots, s_S\}$ to be performed during the week, which is discretised into a set of *time slots* $\mathcal{T} = \{t_1, \ldots, t_T\}$ (e.g. $T = 60$ for 12 slots per day $\times$ 5 working days).

The problem asks to find an assignment of agents to services at specific time slots that satisfies a collection of hard constraints while maximising a multi-criteria objective function composed of:

(i) **Similarity (revenue):** the accumulated expertise reward of assigning agents to the services they are best suited for;

(ii) **Continuity of care:** a preference for having as few distinct agents as possible attending the services of a single user that share the same service type;

(iii) **Working-hour cost:** a penalty incurred whenever an agent works beyond their contractual (normal) hours.

1.1 Input Data

An HCORAP instance is defined by the following data.

- U: number of users; S: number of services; A: number of agents; T: number of time slots.
- $\mathrm{SU}_u \subseteq \mathcal{S}$: the set of services belonging to user u. These form a partition of $\mathcal{S}$.
- $\mathrm{SEQ}_q \subseteq \mathcal{S}$ $(q = 1, \ldots, Q)$: groups of services that belong to the same user *and* have the same service type. Ideally, all services in a SEQ_q should be assigned to the same agent (continuity of care).
- $\mathrm{TSA}_{a,t} \in \{0, 1\}$: availability matrix; $\mathrm{TSA}_{a,t} = 1$ iff agent a is available to work at time slot t.
- $\mathrm{TSS}_{s,t} \in \{0, 1\}$: suitability matrix; $\mathrm{TSS}_{s,t} = 1$ iff service s can be performed at time slot t.
- $r_{a,s} \in \{0, 1, 2, 3, 4\}$: reward (expertise) score of assigning agent a to service s. A value of 0 means the assignment is *forbidden* (the agent lacks the required qualification).
- $\mathrm{HN}_a \in \mathbb{Z}_{\geq 0}$: contractual (normal) weekly working hours for agent a.
- $\mathrm{HE}_a \in \mathbb{Z}_{\geq 0}$: maximum allowed extra working hours for agent a (beyond HN_a).
- $P \in \mathbb{Z}$: penalty weight for each extra working hour (typically $P = -1$).

2 Illustrative Example

Example 1. *Consider a small instance with $U = 2$ users, $A = 2$ agents, $S = 4$ services, and $T = 4$ time slots per week (for illustration purposes).*

Service–user assignment:

$$\mathrm{SU}_1 = \{s_1, s_2\}, \quad \mathrm{SU}_2 = \{s_3, s_4\}.$$

Continuity-of-care groups:

$$\mathrm{SEQ}_1 = \{s_1, s_2\}, \quad \mathrm{SEQ}_2 = \{s_3\}, \quad \mathrm{SEQ}_3 = \{s_4\}.$$

Here s_1 and s_2 are services of the same type for user 1, so they form one sequence.

***Agent availability** (TSA) **and service time windows** (TSS):*

$$\mathrm{TSA} = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 \end{pmatrix}, \quad \mathrm{TSS} = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}.$$

Agent a_1 is available in slots t_1, t_2; agent a_2 in slots t_2, t_3, t_4. Service s_1 can be done in t_1, t_2; service s_2 in t_2, t_3; etc.

***Reward matrix** (r):*

$$r = \begin{pmatrix} 3 & 2 & 0 & 1 \\ 1 & 0 & 4 & 3 \end{pmatrix}.$$

Agent a_1 is forbidden from service s_3 ($r_{1,3} = 0$). Agent a_2 is forbidden from service s_2 ($r_{2,2} = 0$).

Working hours: $\mathrm{HN}_1 = 2$, $\mathrm{HE}_1 = 0$; $\mathrm{HN}_2 = 2$, $\mathrm{HE}_2 = 1$; $P = -1$.

A feasible solution:

Service	*Agent*	*Time slot*	*Reward*
s_1	a_1	t_1	*3*
s_2	a_1	t_2	*2*
s_3	a_2	t_3	*4*
s_4	a_2	t_4	*3*

Analysis:

- Similarity (revenue): $3 + 2 + 4 + 3 = 12$.
- Continuity of care: $\mathrm{SEQ}_1 = \{s_1, s_2\}$ *is served entirely by a_1 (1 agent $\rightarrow$ penalty 0). Total continuity penalty* $= 0$.
- Working-hour cost: *agent a_1 works 2 services* $= \mathrm{HN}_1 \Rightarrow$ *no extra hours. Agent a_2 works 2 services* $= \mathrm{HN}_2 \Rightarrow$ *no extra hours.*
- ***Total objective:*** $12 - 0 - 0 = 12$ *(to be maximised).*

3 Main Constraints

The HCORAP imposes the following hard constraints:

C1. At most one assignment per service. Each service is attended by at most one agent at at most one time slot.

C2. At most one service per agent per time slot. An agent can perform at most one service in any given time slot.

C3. Agent availability. Agent a may only work at time slot t if $\mathrm{TSA}_{a,t} = 1$.

C4. At most one service per user per time slot. A user cannot receive two services simultaneously.

C5. Service coverage. Every service must be attended at some time slot (hard constraint in the basic model; can be relaxed to a soft constraint in the stratified variant).

C6. Maximum working hours. Agent a may work at most $\mathrm{HN}_a + \mathrm{HE}_a$ services in total.

C7. Qualification. Agent a can only be assigned to service s if $r_{a,s} > 0$.

Additionally, the following **soft (optimisation) objectives** are encoded:

O1. Maximise total expertise reward: $\sum_{a\in\mathcal{A}}\sum_{s\in\mathcal{S}} r_{a,s} \cdot y_{a,s}$.

O2. Minimise continuity-of-care violation: For each sequence SEQ_q, minimise the number of distinct agents assigned to its services (ideally 1).

O3. Minimise extra-hour penalty: For each agent a, penalise by $|P|$ for every service beyond HN_a.

4 Integer Linear Programming (ILP) Model

4.1 Decision Variables

$$x_{a,s,t} \in \{0,1\} \qquad = 1 \text{ iff agent } a \text{ performs service } s \text{ at time slot } t \tag{1}$$

$$y_{a,s} \in \{0,1\} \qquad = 1 \text{ iff agent } a \text{ performs service } s \text{ (at any time slot)} \tag{2}$$

$$z_{s,t} \in \{0,1\} \qquad = 1 \text{ iff service } s \text{ is performed at time slot } t \tag{3}$$

$$w_{a,q} \in \{0,1\} \qquad = 1 \text{ iff agent } a \text{ performs some service in } \mathrm{SEQ}_q \tag{4}$$

$$e_a \in \mathbb{Z}_{\geq 0} \qquad \text{number of extra hours worked by agent } a \tag{5}$$

4.2 Objective Function

$$\boxed{\max \underbrace{\sum_{a\in\mathcal{A}}\sum_{s\in\mathcal{S}} r_{a,s} \cdot y_{a,s}}_{\text{expertise reward}} - \underbrace{\sum_{q=1}^{Q}\left(\sum_{a\in\mathcal{A}} w_{a,q} - 1\right)^{+}}_{\text{continuity penalty}} + \underbrace{P \cdot \sum_{a\in\mathcal{A}} e_a}_{\text{extra-hour penalty } (P<0)}} \tag{6}$$

where $(z)^+ = \max(z, 0)$. In practice, the continuity penalty is linearised using auxiliary variables (see the MaxSAT encoding below for a sorting-network approach), or equivalently: for each SEQ_q, minimise $\sum_{a\in\mathcal{A}} w_{a,q}$.

4.3 Constraints

Linking x and y:

$$y_{a,s} = \sum_{t\in\mathcal{T}} x_{a,s,t} \qquad \forall a \in \mathcal{A},\ s \in \mathcal{S} \tag{7}$$

Linking x and z:

$$z_{s,t} = \sum_{a\in\mathcal{A}} x_{a,s,t} \qquad \forall s \in \mathcal{S},\ t \in \mathcal{T} \tag{8}$$

C1 – At most one assignment per service:

$$\sum_{a\in\mathcal{A}}\sum_{t\in\mathcal{T}} x_{a,s,t} \leq 1 \qquad \forall s \in \mathcal{S} \tag{9}$$

C2 – At most one service per agent per time slot:

$$\sum_{s \in \mathcal{S}} x_{a,s,t} \leq 1 \qquad \forall a \in \mathcal{A},\ t \in \mathcal{T} \tag{10}$$

C3 – Agent availability and service suitability:

$$x_{a,s,t} = 0 \qquad \forall (a,s,t) \text{ s.t. } \mathrm{TSA}_{a,t} = 0 \text{ or } \mathrm{TSS}_{s,t} = 0 \text{ or } r_{a,s} = 0 \tag{11}$$

C4 – At most one service per user per time slot:

$$\sum_{s \in \mathrm{SU}_u} z_{s,t} \leq 1 \qquad \forall u \in \mathcal{U},\ t \in \mathcal{T} \tag{12}$$

C5 – Service coverage (hard):

$$\sum_{t \in \mathcal{T}} z_{s,t} \geq 1 \qquad \forall s \in \mathcal{S} \tag{13}$$

C6 – Maximum working hours:

$$\sum_{s \in \mathcal{S}} y_{a,s} \leq \mathrm{HN}_a + \mathrm{HE}_a \qquad \forall a \in \mathcal{A} \tag{14}$$

Extra hours:

$$e_a \geq \sum_{s \in \mathcal{S}} y_{a,s} - \mathrm{HN}_a \qquad \forall a \in \mathcal{A} \tag{15}$$

$$e_a \geq 0 \qquad \forall a \in \mathcal{A}$$

Linking y and w:

$$w_{a,q} \geq y_{a,s} \qquad \forall a \in \mathcal{A},\ q = 1, \ldots, Q,\ s \in \mathrm{SEQ}_q \tag{16}$$

$$w_{a,q} \leq \sum_{s \in \mathrm{SEQ}_q} y_{a,s} \qquad \forall a \in \mathcal{A},\ q = 1, \ldots, Q \tag{17}$$

5 SAT / MaxSAT Model

The paper [1] proposes a **partial weighted MaxSAT** encoding. All hard constraints are encoded as *hard clauses*, while the three optimisation objectives are encoded as *soft clauses* with appropriate weights.

5.1 Boolean Variables

The following Boolean variables are created (only for *valid* combinations, i.e. where $r_{a,s} > 0$, $\mathrm{TSA}_{a,t} = 1$, and $\mathrm{TSS}_{s,t} = 1$):

- **Main variables** $x_{a,s,t}$: true iff agent a performs service s at time slot t.

The following **auxiliary variables** are introduced together with their definition constraints.

- **Assignment variables** $y_{a,s}$: true iff agent a performs service s at some time slot.

- **Service Time-Slot variables** $z_{s,t}$ (`su` in code): true iff service s is performed at time slot t.
- **Sequence Assignment variables** $w_{a,q}$ ($\texttt{ss}_{a,q}$ in the paper, `s` in code): true iff agent a performs at least one service in SEQ_q.
- **Service Count variables** $\hat{w}_{a,i}$ ($w_{a,i}$ in the paper): true iff agent a is assigned at least i services. Defined via a *sorting network* over $\{y_{a,s}\}_s$:

$$\hat{w}_{a,i} \;\Leftrightarrow\; \sum_{s\in\mathcal{S}} y_{a,s} \geq i \qquad \forall a \in \mathcal{A},\; i \in \{1,\ldots,\mathrm{HN}_a + \mathrm{HE}_a + 1\} \tag{18}$$

- **Distinct Agent Count variables** $c_{q,i}$: true iff the services of sequence q are performed by at least i distinct agents. Defined via a *sorting network* over $\{w_{a,q}\}_a$:

$$c_{q,i} \;\Leftrightarrow\; \sum_{a\in\mathcal{A}} w_{a,q} \geq i \qquad \forall q \in \{1,\ldots,|\mathrm{SEQ}|\},\; i \in \{1,\ldots,|\mathrm{SEQ}_q|\} \tag{19}$$

Equations (18) and (19) use **sorting network encoding** [6]: given n input variables $\{x_1,\ldots,x_n\}$, the encoding introduces n output variables $\{o_1,\ldots,o_n\}$ and $O(n\log^2 n)$ hard clauses ensuring that o_i is true iff at least i inputs are true.

5.2 Hard Clauses

Reification (channelling):

$$y_{a,s} \Leftrightarrow \bigvee_{t\in\mathcal{T}} x_{a,s,t} \qquad \text{(encoded via unit implication + one big clause)} \tag{20}$$

$$z_{s,t} \Leftrightarrow \bigvee_{a\in\mathcal{A}} x_{a,s,t} \qquad \text{(similarly)} \tag{21}$$

$$w_{a,q} \Leftrightarrow \bigvee_{s\in\mathrm{SEQ}_q} y_{a,s} \qquad \text{(for } |\mathrm{SEQ}_q| > 1\text{; otherwise } w_{a,q} = y_{a,s}\text{)} \tag{22}$$

At-Most-One (AMO) constraints: These are encoded using standard AMO encodings (e.g. ladder/commander encoding):

$$\mathrm{AMO}\big(\{x_{a,s,t} : a \in \mathcal{A},\, t \in \mathcal{T}\}\big) \qquad \forall s \in \mathcal{S} \quad \text{(C1)} \tag{23}$$

$$\mathrm{AMO}\big(\{x_{a,s,t} : s \in \mathcal{S}\}\big) \qquad \forall a \in \mathcal{A},\; t \in \mathcal{T} \quad \text{(C2)} \tag{24}$$

$$\mathrm{AMO}\big(\{z_{s,t} : s \in \mathrm{SU}_u\}\big) \qquad \forall u \in \mathcal{U},\; t \in \mathcal{T} \quad \text{(C4)} \tag{25}$$

Implicit constraint C3: Handled during variable creation: if $r_{a,s} = 0$, $\mathrm{TSA}_{a,t} = 0$, or $\mathrm{TSS}_{s,t} = 0$, the corresponding $x_{a,s,t}$ variable is set to a constant "false" literal.

Service coverage (C5, hard version):

$$\bigvee_{t\in\mathcal{T}} z_{s,t} \qquad \forall s \in \mathcal{S} \tag{26}$$

Maximum working hours (C6): Encoded using **sorting networks**. For each agent a, the vector of $y_{a,s}$ variables is sorted; then the output at position $\mathrm{HN}_a + \mathrm{HE}_a$ is forced to false:

$$\mathrm{Sort}\big(\{y_{a,s} : s \in \mathcal{S},\, r_{a,s} > 0\}\big) \implies \neg\,\mathrm{out}[\mathrm{HN}_a + \mathrm{HE}_a] \qquad \forall a \in \mathcal{A} \tag{27}$$

5.3 Soft Clauses (Objective Function)

The objective function to maximise consists of three components:

$$\text{objective} = \underbrace{\text{similarity}}_{\text{(O1)}} + \underbrace{\text{stability}}_{\text{(O2)}} - \underbrace{\text{cost}}_{\text{(O3)}} \tag{28}$$

where:

$$\text{similarity} = \sum_{a \in \mathcal{A}} \sum_{s \in \mathcal{S}} r_{a,s} \cdot y_{a,s} \tag{29}$$

$$\text{stability} = \sum_{q=1}^{|\text{SEQ}|} \sum_{i=1}^{|\text{SEQ}_q|} \left(1 - c_{q,i}\right) \tag{30}$$

$$\text{cost} = \sum_{a \in \mathcal{A}} \sum_{i=\text{HN}_a+1}^{\text{HN}_a+\text{HE}_a} |P| \cdot \hat{w}_{a,i} \tag{31}$$

Each component is encoded using *weighted soft clauses* $\langle c, w \rangle$, where $w > 0$ is the reward for satisfying clause c. MaxSAT maximises the total weight of satisfied soft clauses while satisfying all hard clauses.

O1 – Similarity (Eq. 29): For each valid pair (a, s) with $r_{a,s} > 0$:

$$\left\langle y_{a,s},\ \ r_{a,s} \right\rangle \qquad \forall a \in \mathcal{A},\ \forall s \in \mathcal{S} \tag{32}$$

O2 – Stability (Eq. 30): The stability term rewards each "unused agent slot" in a sequence. Using the sorting network output variables $c_{q,i}$ (Eq. 19):

$$\left\langle \neg\, c_{q,i},\ \ 1 \right\rangle \qquad \forall q \in \{1, \ldots, |\text{SEQ}|\},\ \forall i \in \{1, \ldots, |\text{SEQ}_q|\} \tag{33}$$

Satisfying $\neg c_{q,i}$ (i.e. fewer than i agents in sequence q) earns a reward of 1. This effectively penalises each additional distinct agent.

O3 – Cost (Eq. 31): MaxSAT requires all soft-clause weights to be *positive*. Since cost subtracts from the objective, we apply the standard transformation:

$$-\text{cost} = -\sum_{a,\,i} |P| \cdot \hat{w}_{a,i} = \underbrace{\sum_{a,\,i} |P|}_{\text{constant}} - \sum_{a,\,i} |P| \cdot (1 - \hat{w}_{a,i}) - \underbrace{\sum_{a,\,i} |P|}_{\text{cancel}} \tag{34}$$

The constant terms can be precomputed and added as an offset. The variable part yields soft clauses with positive weights:

$$\left\langle \neg\, \hat{w}_{a,i},\ \ |P| \right\rangle \qquad \forall a \in \mathcal{A},\ \forall i \in \{\text{HN}_a + 1, \ldots, \text{HN}_a + \text{HE}_a\} \tag{35}$$

Satisfying $\neg \hat{w}_{a,i}$ (agent a works fewer than i services) earns a reward of $|P|$, thus reducing the cost.

5.4 Stratified Variant

In the **stratified** encoding, constraint C5 (service coverage) becomes a *soft* clause with a weight high enough to be strictly prioritised over the other objectives:

$$\left(\bigvee_{t \in \mathcal{T}} z_{s,t},\ \ \text{weight} = W_{\text{total}} + 1 \right) \qquad \forall s \in \mathcal{S} \tag{36}$$

where W_{total} is the sum of all other soft-clause weights. This creates a lexicographic optimisation: first maximise the number of covered services, then maximise the original objective.

6 Optimisation Approaches

The HCORAP is a **Constrained Optimisation Problem (COP)**: find an assignment satisfying all hard constraints (Section 3) that *maximises* the weighted objective (Equation 6). Since standard SAT solvers handle only satisfiability, optimisation must be achieved by a meta-algorithm that orchestrates one or more SAT calls. In what follows we present three paradigms—*Non-Incremental SAT*, *Incremental SAT*, and *MaxSAT*—and compare their trade-offs in the context of HCORAP.

We write φ for the conjunction of all hard clauses (Section 5), W for the total soft-clause weight budget, and $\mathrm{obj}(\sigma)$ for the objective value of a satisfying assignment σ. A *bounded* SAT query $\mathrm{SAT}(\varphi, \mathrm{obj} \geq K)$ asks whether there exists a model of φ whose objective is at least K. This is realised by augmenting φ with a **cardinality / pseudo-Boolean constraint** that enforces the bound.

6.1 Non-Incremental SAT

In the non-incremental approach, a **fresh SAT solver instance** is created for every query. The formula φ and the cardinality encoding for the current bound are regenerated from scratch each time.

6.1.1 Top-Down Linear Search

Start from any feasible solution and iteratively tighten the lower bound until the problem becomes unsatisfiable.

Algorithm 1 Non-Incremental Top-Down Linear Search (maximisation)

```
σ* ← SAT(φ)                                   ▷ find any feasible solution
K ← obj(σ*)
while SAT(φ ∧ obj ≥ K+1) do                   ▷ fresh solver each call
    σ* ← model;  K ← obj(σ*)
end while
return σ*                                      ▷ optimal
```

Number of SAT calls: $\mathrm{obj}^* - \mathrm{obj}_0 + 2$, where obj_0 is the first solution's value. In the worst case this can be $O(W)$, making it impractical for large W.

6.1.2 Bottom-Up Linear Search

Start from $K = W$ (maximum possible objective) and decrease until satisfiable.

Algorithm 2 Non-Incremental Bottom-Up Linear Search (maximisation)

```
K ← W
while ¬SAT(φ ∧ obj ≥ K) do                    ▷ fresh solver
    K ← K − 1
end while
return model                                  ▷ optimal with obj = K
```

Number of SAT calls: $W - \mathrm{obj}^* + 1$. Efficient when obj^* is close to W; poor otherwise.

6.1.3 Binary (Dichotomic) Search

The most efficient non-incremental strategy. Maintain an interval $[lb, ub]$ and probe the midpoint.

Algorithm 3 Non-Incremental Binary Search (maximisation)

```
lb ← 0;   ub ← W;   σ* ← None
while lb ≤ ub do
    mid ← ⌈(lb + ub)/2⌉
    if SAT(φ ∧ obj ≥ mid) then                    ▷ fresh solver
        σ* ← model;   lb ← obj(σ*)                ▷ tighten using actual obj. value
    else
        ub ← mid − 1
    end if
end while
return σ*                                          ▷ optimal
```

Number of SAT calls: $O\big(\log_2(W)\big)$. For a typical HCORAP instance with $W \approx 600$, this gives $\lceil \log_2 600 \rceil = 10$ calls.

6.1.4 Cardinality Encoding

The bound obj $\geq K$ is enforced by encoding the (weighted) sum of soft-clause satisfaction as a cardinality or pseudo-Boolean constraint. Common encodings include [4, 5]:

- **Sequential counter** [4]: $O(n \cdot K)$ auxiliary variables and clauses. Compact with good unit-propagation properties.
- **Totalizer** [5]: $O(n \log n)$ variables, $O(n \log^2 n)$ clauses. Better for large n.
- **Sorting networks** [6]: based on odd–even merge, balances encoding size and propagation strength. *This is the encoding used in the HCORAP implementation.*

In the non-incremental setting, the cardinality encoding is **rebuilt from scratch** for each SAT call, which is the main source of overhead.

6.1.5 Key Properties

Property	**Non-Incremental SAT**
Learned clauses reused across iterations	No
Cardinality encoding rebuilt each call	Yes
Best search strategy	Binary search
Number of SAT calls (binary)	$O(\log W)$
Implementation complexity	Low

6.2 Incremental SAT

In the incremental approach, a **single solver instance** is kept alive across all iterations. The solver retains its *learned-clause database*, which can dramatically speed up subsequent queries since conflict analysis from earlier iterations remains valid.

The key technical challenge is modifying the objective bound between calls without invalidating the solver state. Two mechanisms are available:

6.2.1 Assumption-Based Bound Tightening

Modern SAT solvers support the **IPASIR** interface [7], which allows passing *assumption literals* to each Solve call. Assumptions are temporary unit-clause assertions that are active only for the current call; they do not permanently alter the clause database.

Idea. Encode the sorting network / cardinality constraint **once** at initialisation. Let $o_0, o_1, \ldots, o_n$ be the sorted output variables, where $o_k = \mathit{true}$ means "at least $k{+}1$ soft clauses are satisfied". To enforce obj $\geq K$, simply pass assumption $o_{K-1} = \mathit{true}$ (or equivalently, for "obj $\leq K$", assume $\neg\, o_K$).

Algorithm 4 Incremental Binary Search with Assumptions (maximisation)

```
solver ← NewSolver(φ)
Build sorting network once; let o_0, ..., o_n be output variables
lb ← 0;  ub ← W;  σ* ← None
while lb ≤ ub do
    mid ← ⌈(lb + ub)/2⌉
    assumptions ← {o_{mid−1}}                    ▷ enforce obj ≥ mid
    if solver.Solve(assumptions) then
        σ* ← model;  lb ← obj(σ*)
    else
        ub ← mid − 1
    end if
end while
return σ*
```

Advantages:

- The cardinality encoding is built **once**, not rebuilt each call.
- All learned clauses from previous iterations remain in the solver.
- Assumptions add zero permanent clauses—the solver's state stays clean.
- Typically **2–10× faster** than non-incremental binary search on the same instance.

6.2.2 Clause-Addition Linear Search

An alternative incremental approach: after finding a solution with obj $= K$, add a *permanent* clause (obj $\geq K{+}1$) to the solver and re-solve.

Algorithm 5 Incremental Linear Search with Clause Addition (maximisation)

```
solver ← NewSolver(φ)
while solver.Solve() do
    σ* ← model;  K ← obj(σ*)
    solver.AddClauses(Encode(obj ≥ K+1))
end while
return σ*                                        ▷ last SAT model is optimal
```

Caveat: Added clauses are *permanent* and cannot be retracted. Old (subsumed) cardinality constraints accumulate in the clause database, increasing memory usage. The assumption-based approach (Algorithm 4) avoids this issue entirely.

6.2.3 Application to HCORAP

The HCORAP codebase implements incremental solving through the `DicoOptimizer` class, which supports both modes:

- When `useAssumptions = false`: a fresh encoding is created per call (non-incremental binary search, Section 6.1).

- When `useAssumptions = true`: the solver is initialised once via `initAssumptionOptimization`, and subsequent calls use `checkSATAssuming` (incremental binary search, Algorithm 4).

The sorting-network outputs serve as the assumption handles: assuming a sorted output to be true or false directly controls the objective bound.

6.2.4 Key Properties

Property	Assumption-based	Clause-addition
Learned clauses reused	Yes	Yes
Encoding built once	Yes	No (rebuilt each tightening)
Subsumed clause accumulation	None	Yes (growing database)
Best search strategy	Binary search	Linear search

6.3 MaxSAT

The most natural approach for HCORAP: encode the problem directly as a **partial weighted MaxSAT** instance and delegate the entire optimisation to a specialised MaxSAT solver.

6.3.1 Formulation

A *partial weighted MaxSAT* (PWMS) instance consists of:

- **Hard clauses** φ_H: must be satisfied (weight $= \infty$).
- **Soft clauses** $\{(c_i, w_i)\}_{i=1}^{m}$: each has a finite weight $w_i > 0$. The solver maximises $\sum_{i:c_i \text{ satisfied}} w_i$.

For HCORAP (Section 5), the mapping is direct:

- φ_H = all hard clauses (reification, AMO, coverage, max-hours).
- Soft clauses = expertise reward (O1) + continuity penalty (O2) + extra-hour penalty (O3), with respective weights.

The MaxSAT solver finds an assignment σ^* that satisfies all hard clauses and maximises the total weight of satisfied soft clauses in a **single invocation**—no manual binary/linear search is needed.

6.3.2 Internal Algorithms of MaxSAT Solvers

State-of-the-art MaxSAT solvers use one of the following internal strategies:

Core-Guided (OLL / RC2). The solver iteratively calls a SAT oracle. When an UNSAT result is obtained, the solver extracts an *unsatisfiable core*—a minimal subset of soft clauses that cannot all be satisfied simultaneously. *Relaxation variables* are added to the core clauses, and a new cardinality constraint bounds how many of them can be relaxed. This process repeats until a SAT result is found [8, 9].

The core-guided approach is particularly effective when the optimum cost (total weight of violated soft clauses) is small relative to the total weight, as few cores need to be extracted.

Linear SAT-UNSAT Search. Internally equivalent to the top-down linear search (Algorithm 1), but executed within the solver with full learned-clause reuse.

Stratified / Weight-Aware. For weighted instances, some solvers partition soft clauses by weight and solve them in decreasing weight order (stratification), which naturally handles the lexicographic structure of the HCORAP stratified variant (Equation 36).

6.3.3 Stratified Optimisation in HCORAP

The HCORAP stratified encoding (Section 5) assigns coverage soft clauses a weight $W_{\text{total}} + 1$ (where W_{total} is the sum of all other soft-clause weights). This guarantees that the MaxSAT solver first maximises service coverage, and only then optimises the secondary objective (expertise reward − continuity penalty − extra-hour cost). This is equivalent to a **two-level lexicographic** MaxSAT problem, which weight-aware solvers handle natively.

6.3.4 DIMACS WCNF Format

The HCORAP tool `hcorap2sat` outputs instances in the **weighted DIMACS CNF (WCNF)** format [3], which is the standard input format for MaxSAT solvers:

```
p wcnf <#vars> <#clauses> <top_weight>
<top_weight> <lit_1> <lit_2> ... 0       (hard clause)
<weight>     <lit_1> <lit_2> ... 0       (soft clause)
```

Any off-the-shelf MaxSAT solver (e.g. WMaxCDCL, RC2, CASHWMaxSAT, EvalMaxSAT, Open-WBO) can then be used to solve the instance.

6.3.5 Key Properties

Property	**MaxSAT**
Explicit bound iteration by user	No (solver-internal)
Objective encoding	Soft clauses with weights
Learned clauses reused	Yes (internally)
Optimality guarantee	Yes (solver proves optimal)
Handles weighted objectives	Yes (natively)
Handles stratified / lexicographic	Yes (via weight hierarchy)
Implementation complexity	Low (single solver call)

6.4 Comparison of Approaches

Table 1: Comparison of optimisation approaches for HCORAP.

Criterion	**Non-Incremental SAT**	**Incremental SAT**	**MaxSAT**
Number of SAT calls	$O(\log W)$ (binary)	$O(\log W)$ (binary)	1 (internal iteration)
Learned-clause reuse	No	Yes	Yes
Encoding rebuilt?	Each call	Once	Once
Multi-component weighted obj.	Manual (encode sum)	Manual (encode sum)	Native (soft weights)
Stratified / lexicographic	Manual (two-phase)	Manual (two-phase)	Native (weight hierarchy)
Solver infrastructure	Any SAT solver	IPASIR-compatible	MaxSAT solver
Implementation effort	Low	Medium	Low
Optimality guarantee	Yes (exhaustive search)	Yes (exhaustive search)	Yes (solver-certified)

Discussion. For HCORAP specifically, the **MaxSAT approach** is the most natural fit because:

(a) The problem already has a multi-component weighted objective that maps directly to weighted soft clauses.

(b) The stratified variant (lexicographic coverage $\succ$ revenue) is handled natively by weight-aware MaxSAT solvers.

(c) No manual search loop is needed; the solver handles all internal iteration, core extraction, and optimality certification.

(d) The HCORAP tool can output standard WCNF files, allowing any state-of-the-art MaxSAT solver to be used as a black box.

The **incremental SAT approach** (assumption-based binary search) is a strong alternative when a MaxSAT solver is not available or when finer control over the search process is desired (e.g. for anytime behaviour or custom heuristics).
The **non-incremental SAT approach** provides the simplest baseline implementation—useful for prototyping and for instances small enough that the overhead of rebuilding the encoding is negligible.

7 Dataset

7.1 Overview

The benchmark dataset used in [1] consists of **400 randomly generated instances** modelling realistic home care scenarios in the Barcelona metropolitan area.

Each instance is characterised by three parameters:

- $U \in \{30, 40\}$: number of users,
- $A \in \{10, 15, 20, 25\}$: number of agents,
- $V \in \{4, 5\}$: number of services per user (so $S = U \times V$).

This gives $2 \times 4 \times 2 = 16$ parameter configurations. For each configuration, **50 instances** are generated, yielding $16 \times 50 = 800$ total in two groups (U30 and U40), with **400 instances per user count**.

7.2 Instance Generation

The instances are generated by the Python script `create_instances.py` [2], which simulates realistic home care data:

- **Agents** are characterised by age (25–60), qualifications (drawn from {basic, nurse, CPR, physio, doctor}), gender, languages, race, and geographic location (GPS coordinates from Barcelona). Working schedules (TSA) are generated as contiguous blocks of 4–12 hours per day, repeated over 5 working days.
- **Users** are elderly individuals (age 70–90) with demographic attributes (gender, language, race, location).
- **Services** are assigned to users (V services per user) with types drawn from {basic (more frequent), nurse, CPR, physio, doctor}. Each service has a time window of 2–5 consecutive time slots on one day of the week.

- **Reward matrix** (r): a similarity score between each agent–service pair is computed based on qualification match, shared demographics (gender, race, language), and geographic distance. The raw scores are then discretised into quartiles, yielding values in $\{1, 2, 3, 4\}$ for valid assignments and 0 for forbidden ones (when the agent lacks the required qualification).
- **Continuity-of-care groups** (SEQ): services for the same user with the same type are grouped into sequences.
- **Working hours:** $\mathrm{HN}_a = \min(\text{total available slots}, 35)$ and $\mathrm{HE}_a = \max(\text{total available slots} - 35, 0)$. The penalty $P = -1$.
- **Time slots:** $T = 12 \times 5 = 60$ (12 one-hour slots per day, 5 days per week).

7.3 Instance File Format

Instances are stored in a plain-text format with the following sections:

Section	Content
`#U`	Number of users U
`#S`	Number of services S
`#A`	Number of agents A
`#TS`	Number of time slots T
`#SU`	Service groups per user (one line per user)
`#SEQ`	Continuity-of-care sequences (one line per group)
`#TSA(i)`	Agent availability matrix ($A \times T$)
`#TSS(i)`	Service time-slot suitability matrix ($S \times T$)
`#r(i,j)`	Reward matrix ($A \times S$)
`#P`	Extra-hour penalty
`#HN(i)`	Normal working hours per agent
`#HE(i)`	Extra working hours per agent

7.4 Instance Statistics

Acknowledgements

The HCORAP benchmark dataset and MaxSAT encoding are provided by Jordi Coll Caballero et al. and are publicly available on GitHub [2].

References

[1] Irene Unceta, Bernat Salbanya, Jordi Coll, Mateu Villaret, and Jordi Nin. Optimizing Resource Allocation in Home Care Services using MaxSAT. *Artificial Intelligence*, 339:104258, 2025.

[2] Jordi Coll Caballero. HCORAP – Home Care Optimised Resource Allocation Problem. `https://github.com/jordicollcaballero/HCORAP`, 2024.

[3] MaxSAT Evaluations. `https://maxsat-evaluations.github.io/2022/rules.html`, 2022.

[4] Carsten Sinz. Towards an Optimal CNF Encoding of Boolean Cardinality Constraints. In *Proceedings of the 11th International Conference on Principles and Practice of Constraint Programming (CP)*, pp. 827–831, Springer, 2005.

[5] Olivier Bailleux and Yacine Boufkhad. Efficient CNF Encoding of Boolean Cardinality Constraints. In *Proceedings of the 9th International Conference on Principles and Practice of Constraint Programming (CP)*, pp. 108–122, Springer, 2003.

Table 2: Summary of instance configurations.

U	A	V	$S = U \times V$	T	# Instances
30	10	4	120	60	50
30	10	5	150	60	50
30	15	4	120	60	50
30	15	5	150	60	50
30	20	4	120	60	50
30	20	5	150	60	50
30	25	4	120	60	50
30	25	5	150	60	50
40	10	4	160	60	50
40	10	5	200	60	50
40	15	4	160	60	50
40	15	5	200	60	50
40	20	4	160	60	50
40	20	5	200	60	50
40	25	4	160	60	50
40	25	5	200	60	50
Total					**800**

[6] Roberto Asín, Robert Nieuwenhuis, Albert Oliveras, and Enric Rodríguez-Carbonell. Cardinality Networks: A Theoretical and Empirical Study. *Constraints*, 16(2):195–221, 2011.

[7] Tomáš Balyo, Armin Biere, Markus Iser, and Carsten Sinz. SAT Race 2015. *Artificial Intelligence*, 241:45–65, 2016. (Defines the IPASIR incremental SAT solver interface.)

[8] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. RC2: An Efficient MaxSAT Solver. *Journal on Satisfiability, Boolean Modeling and Computation*, 11(1):53–64, 2019.

[9] Antonio Morgado, Carmine Dodaro, and Joao Marques-Silva. Core-Guided MaxSAT with Soft Cardinality Constraints. In *Proceedings of the 20th International Conference on Principles and Practice of Constraint Programming (CP)*, pp. 564–573, Springer, 2014.